

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Rhushabh Vaghela	India	rhushabhvaghela@gmail.com	
Robert Nkuna	South Africa	nkunarobert60@gmail.com	
Israel Olayinka Shodiya			X

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).

Team member 1	Rhushabh Vaghela
Team member 2	Robert Nkuna
Team member 3	

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

We initially connected with Israel via the WQU Group Discussion forum, where he agreed to the project distribution and provided his number to transition to WhatsApp for real-time collaboration. Israel successfully joined our WhatsApp group but subsequently ceased all communication. Despite multiple direct follow-ups in the group chat tagging him for status updates on his assigned work (Categories 1 and 3) and providing the shared Google Drive link, he never replied or contributed any material. As a result, we have marked him as a non-contributing member and submitted this project as a two-person team covering 5 categories, in strict accordance with the rubric guidelines. I have shared the Group Forum Conversations as well as WhatsApp Group Chats within the zip file for the proofs.

Report

Step 1: Basics

Category 2: K-means Clustering:

Unsupervised by design, K-Means clusters unlabelled records by gathering them around K central points, or centroids. The algorithm assigns each observation to the nearest cluster centre, then recalculates the centroid locations based on the average coordinates of the assigned points, repeating this loop until the boundaries stabilize.

Category 4: Classification Trees:

Operating as a supervised classifier, a decision tree recursively splits a sample into smaller fragments by applying a series of logical, sequential rules. It keeps branching until every record receives a final class label.

Category 5: Linear Discriminant Analysis (LDA)

LDA classifies labelled data by projecting it to a lower-dimensional axis. The projection is mathematically designed to maximize the distance between class means while keeping the variation within each class to a minimum.

Category 6: Support Vector Machine (SVM)

An SVM is a supervised classifier that searches for the geometric hyperplane separating the classes with the widest possible gap. It specifically optimizes the margin, which represents the separation between the boundary and the nearest class representatives.

Category 7: Neural Networks

Modelled loosely on biological neural pathways, a neural network processes data through networks of calculation nodes. Information flows from the input step through hidden layers that apply non-linear mathematical transformations, finally producing a prediction at the output layer.

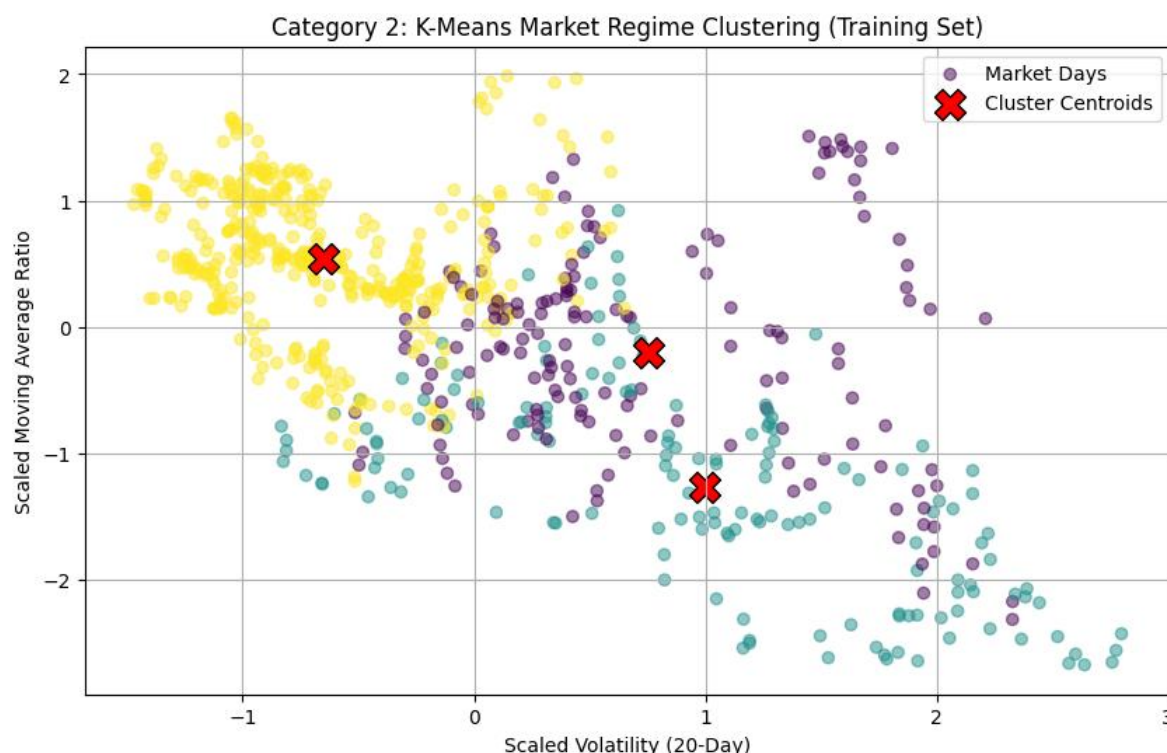
Step 2: Individual Contribution

Category 2: K-means Clustering

Advantages:

K-Means runs quickly and is easy to configure, scaling well to large datasets. Financial desks commonly use it for client segmentation, identifying shifts in market regimes, or finding assets with similar return profiles.

Computation:



Interpretation and Recommended Action: The scatter plot shows that K-Means divided the historical data into three regimes based on volatility and moving-average ratios, without reference to actual price direction. Since calculations rely on Euclidean distance, the features were scaled beforehand so that the larger MA-ratio values would not overshadow the smaller volatility figures. In a live setting, these cluster labels should not drive buy/sell decisions on their own. We recommend feeding them into supervised models as a categorical feature (for example, tagging an observation as 'high-volatility regime') so that risk parameters can shift with the detected regime.

The implementation code and outputs for this model are in the accompanying Jupyter notebook.

Disadvantages:

Manually selecting the K parameter in advance remains a requirement. The final groupings are sensitive to where the centroids are initially placed, meaning the algorithm can settle on local optimums. It also performs poorly with outliers or non-spherical clusters.

Equations:

The objective is to minimise the within-cluster sum of squares, defined in the modules as $J =$

$\sum_{j=1}^{\{k\}} \sum_{i=1}^{\{n\}} \|x_i^j - c_j\|^2$. Centroids are updated each iteration by calculating the mean of the cluster: $v_i = (\frac{1}{c_i}) \sum_{j=1}^{\{c_i\}} x_j$.

Features:

K-Means is an unsupervised, distance-based method. It assigns each point to exactly one cluster using Euclidean distance, so cluster memberships are mutually exclusive.

Guide:

- **Inputs:** an unlabelled numerical dataset, the target number of clusters (K), and a distance metric.
- **Outputs:** a cluster label for each observation and the final centroid coordinates.

Hyperparameters:

n_clusters (target number of groups), init (initialisation method, typically k-means++), n_init (how many times the algorithm starts from different initial centroid positions), and max_iter (iteration cap per run).

Illustration:

A two-dimensional scatter plot of the scaled validation data. Points are coloured by cluster membership, and large red "X" markers show the final centroid positions.

Journal:

Ammann, Manuel, and Michael Verhofen. "The Clustering of Hedge Funds." *Journal of Banking & Finance*, vol. 30, no. 11, 2006, pp. 3031–3059.

Keywords:

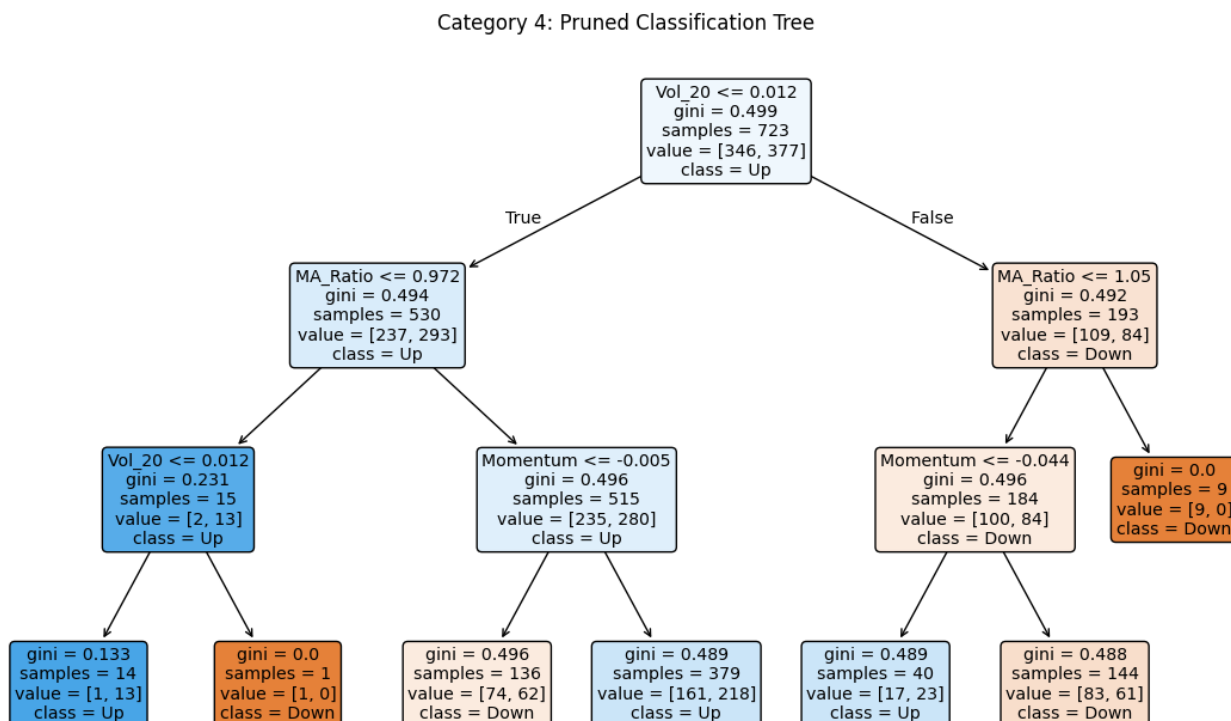
#Centroids, #UnsupervisedLearning, #EuclideanDistance, #ElbowPlot, #WithinSumofSquares(WSS).

Category 4: Classification Trees

Advantages:

The standout feature of decision trees is how easily they can be interpreted. You can map out the entire decision path as a flowchart on a whiteboard to explain a trade flag to a risk committee. They also fit non-linear curves naturally, bypassing the need for standardizing or scaling the inputs beforehand.

Computation:



Interpretation & Recommended Action:

The visual plot confirms the tree successfully splits based on Gini impurity, favoring volatility and momentum rules. However, the accuracy hovers near the 0.50 baseline, confirming the structural weakness of single trees in noisy financial environments. For live trading, we recommend utilizing this logic purely as a preliminary signal filter rather than a standalone capital allocation tool.

The implementation code and outputs for this model are in the accompanying Jupyter notebook.

Disadvantages:

An unconstrained decision tree will quickly overfit, memorizing noise in the training set and failing on out-of-sample data. They are also highly unstable; slight shifts in the training set can produce an entirely different model structure.

Equations:

The algorithm chooses each split by minimising Gini impurity, calculated as $Gini = 1 - \sum_{k=1}^K p_k^2$, where p_k is the proportion of items belonging to class k in a given node.

Features:

Trees create axis-parallel decision boundaries. Because they are non-parametric, no distributional assumption is needed. Pruning (removing unnecessary branches after training) is a standard step to improve generalisation.

Guide:

- **Inputs:** categorical or continuous numerical features (e.g., missed-payment counts, credit-history length, or moving averages).
- **Outputs:** a discrete class prediction (e.g., Default vs. Non-Default) and the probability associated with the terminal node.

Hyperparameters:

The main parameters to tune are `max_depth` (maximum tree depth), `min_samples_split` (minimum observations needed to create a new branch), and `min_samples_leaf` (minimum observations allowed in a terminal node).

Illustration:

A top-down flowchart showing the decision rules. Each node displays the splitting condition (e.g., "Credit History > 2 Years"), the Gini impurity score, and the majority class. Depth increases from root to leaves.

Journal:

Adcock, C. J., and N. Meade. "Using Parametric Classification Trees for Model Selection with Applications to Financial Risk Management." *European Journal of Operational Research*, vol. 259, no. 2, 2017, pp. 746-765.

Keywords:

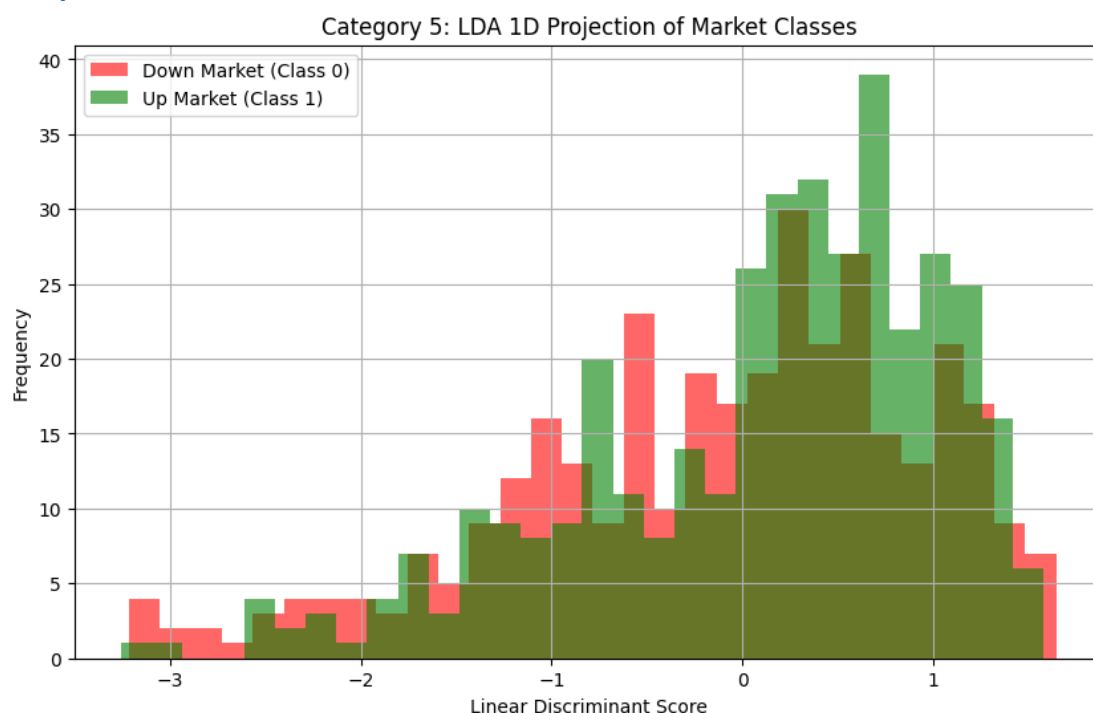
#GiniImpurity, #RootNode, #LeafNode, #Pruning, #DecisionBoundary.

Category 5: Linear Discriminant Analysis (LDA)

Advantages:

LDA is computationally light, offering fast training and prediction alongside clear coefficients. Since it also works as a dimensionality reduction technique, it has been a popular choice for credit scoring and forecasting corporate bankruptcy ever since Altman's 1968 study.

Computation:



Interpretation and Recommended Action: The histogram illustrates how LDA projects multi-dimensional data onto a single axis to separate the 'Up' and 'Down' classes. However, the heavy overlap between the two distributions shows that financial returns violate LDA's assumption of normality and equal covariance. As a result, the model's binary classifications are unreliable. We recommend using the continuous probability estimates (`predict_proba`) instead, setting a high threshold such as 65% before initiating a trade.

The implementation code and outputs for this model are in the accompanying Jupyter notebook.

Disadvantages:

LDA relies on the assumption of normally distributed features and equal covariance matrices across groups. Because financial returns typically feature fat tails and volatility clustering, these assumptions rarely hold in practice. The model is also vulnerable to outliers.

Equations:

The model computes the class mean as $\mu_k = \left(\frac{1}{N_k}\right) \sum x_i$. It then evaluates the discriminant function $\delta_k(x) = x^T \Sigma_k^{-1} \mu_k - 0.5 \mu_k^T \Sigma_k^{-1} \mu_k + \ln(\pi_k)$ and assigns the observation to

Group Number: 15889

whichever class yields the highest score.

Features:

LDA is a supervised, probabilistic classifier that draws linear decision boundaries. It uses Bayes' theorem to estimate the conditional probability that a new observation belongs to each target class.

Guide:

- **Inputs:** a scaled feature matrix (X) and discrete class labels (y).
- **Outputs:** predicted class labels, class probabilities, and the discriminant components.

Hyperparameters:

solver (computation method, e.g., singular value decomposition or least squares), shrinkage (regularisation term for the covariance estimate), and `n_components` (number of discriminant axes to keep).

Illustration:

A stacked frequency histogram of the one-dimensional projection along the discriminant axis. The two semi-transparent distributions ("Up" and "Down" classes) overlap, showing how well (or poorly) the algorithm separates the targets.

Journal:

Altman, Edward I. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *The Journal of Finance*, vol. 23, no. 4, 1968, pp. 589–609.

Keywords:

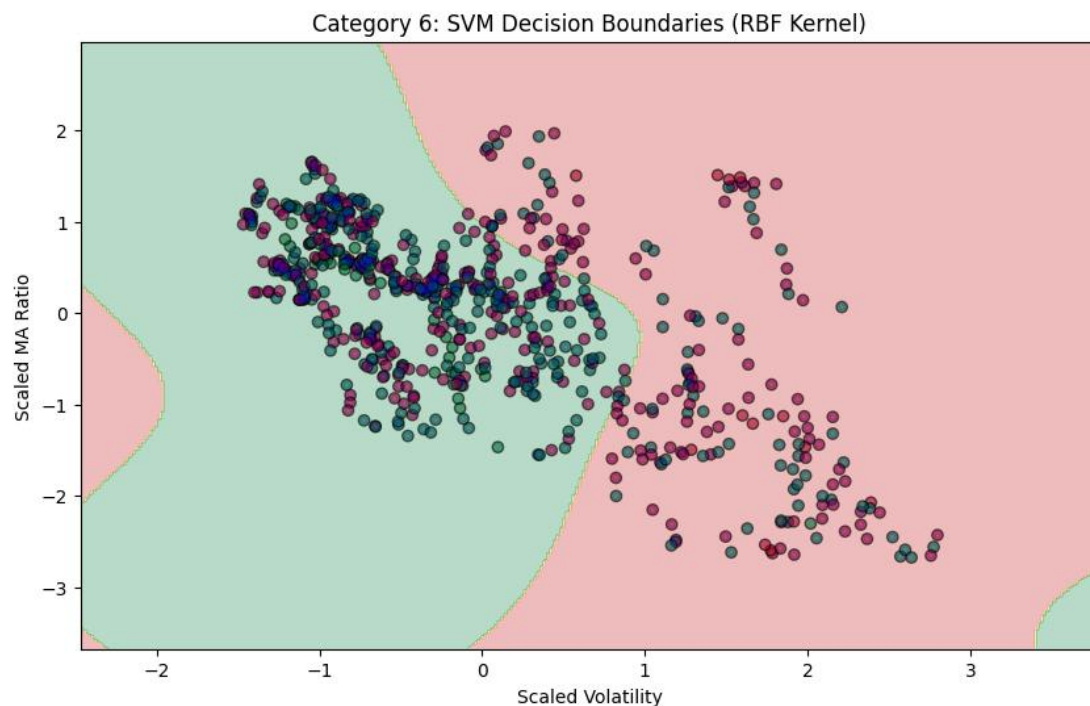
#DimensionalityReduction, #Between-ClassVariance, #Within-ClassScatter, #Bayes'Theorem, #DiscriminantFunction.

Category 6: Support Vector Machine (SVM)

Advantages:

SVMs are particularly effective when the input features are high-dimensional, such as when running a model with dozens of technical indicators. They are also memory-efficient because the final boundary depends only on a small subset of training points called the support vectors.

Computation:



SVM Test Set Accuracy: 0.5041

Number of Support Vectors: 663 out of 723

Interpretation & Recommended Action:

The SVM decision boundary plot reveals a complex, non-linear relationship between volatility and the moving average ratio. With 663 support vectors out of 723 training samples, the model is utilizing nearly 92% of the data to define its margins, which explains the 50.41% test accuracy as the model is struggling to find a clear, generalized signal amidst the market noise. Because the 'Support Vectors' (the blue outlined points) are so densely packed across the decision regions, the current feature set does not allow for a clear geometric separation of 'Up' vs 'Down' days. We recommend the execution desk use this SVM model only as a secondary volatility filter. Specifically, avoid taking aggressive positions when the market state falls near the decision boundaries (the transition zones between red and green), as these represent areas of high uncertainty.

The implementation code and outputs for this model are in the accompanying Jupyter notebook.

Group Number: 15889

Disadvantages:

The main drawback is training time. SVMs scale poorly with dataset size, which becomes a problem on large tick-level datasets. They also do not produce probability estimates by default; obtaining probabilities requires a secondary calibration step such as Platt scaling.

Equations:

The linear separating boundary is defined by $\beta_0 + \sum_{j=1}^p \beta_j X_j = 0$ For non-linear cases, the

Radial Basis Function (RBF) kernel is used, defined as $K(X_i, X_{i'}) = \exp\left(-\gamma \|X_i - X_{i'}\|^2\right)$.

Features:

The model uses a soft-margin formulation. Slack variables allow a controlled number of misclassifications, which prevents the boundary from warping around outliers. Through the kernel trick, the algorithm maps data into a higher-dimensional space to handle non-linear problems without computing the transformed coordinates directly.

Guide:

- **Inputs:** standardised numerical variables (e.g., z-scored moving-average ratios or normalised yield spreads). Feature scaling is required.
- **Outputs:** a class label (e.g., "Up" or "Down" market trend).

Hyperparameters:

The parameters to optimise are C (the regularisation penalty controlling the soft margin), the kernel type (linear, polynomial, or RBF), and γ (which sets how far the influence of a single training example extends).

Illustration:

A two-dimensional scatter plot with "Trailing Moving Average Ratio" on the x-axis and "Relative Strength Index" on the y-axis, both scaled from -3 to 3. Blue and red dots mark the two market classes. A solid line shows the separating hyperplane, and dashed lines parallel to it pass through the support vectors.

Journal:

"Non-Linear Stock Market Prediction with Support Vector Machines." International Research Journal on Advanced Engineering Hub, vol. 3, no. 3, 2025, pp. 615-621.

Keywords:

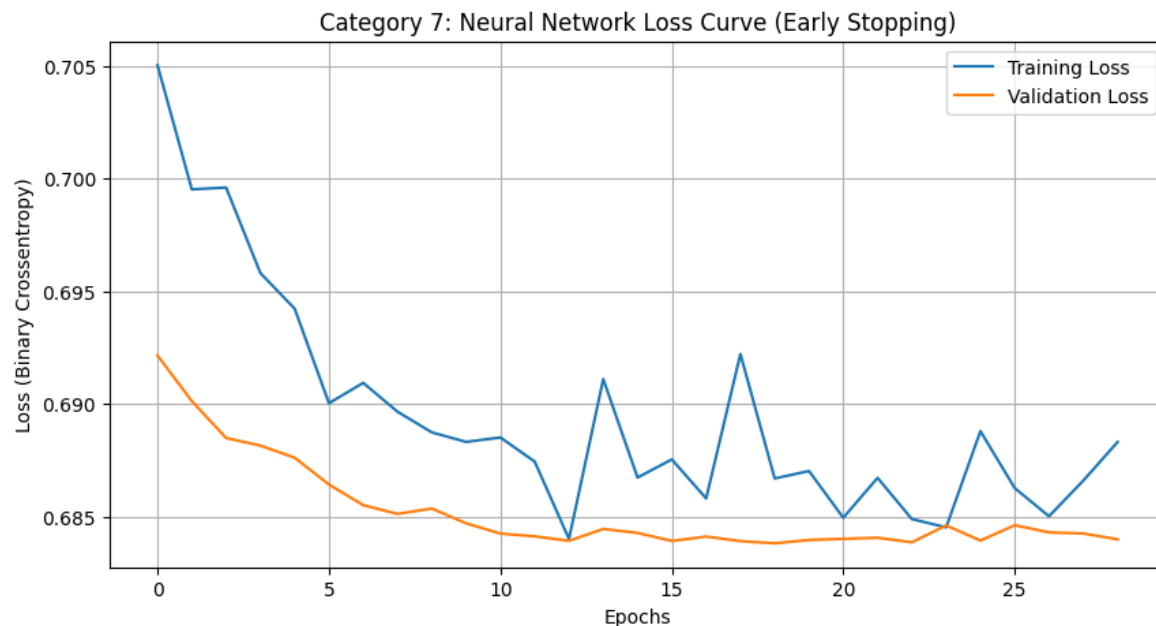
#Hyperplane, #MaximalMargin, #SupportVectors, #KernelTrick, #SlackVariables.

Category 7: Neural Networks

Advantages:

Neural networks are highly effective at learning non-linear, complex patterns directly from raw data. Because the hidden layers learn representations automatically, they reduce the time analysts spend hand-crafting features.

Computation:



Interpretation and Recommended Action: The loss curve displays the training progression. Using a separate validation fold to trigger early stopping allowed us to halt training before the network begins overfitting. The final performance was evaluated on a clean test set. Because neural networks function as black boxes, we advise setting strict stop-loss orders on any live trades initiated by the model's predictions.

The implementation code and outputs for this model are in the accompanying Jupyter notebook.

Disadvantages:

The lack of transparency makes neural networks hard to audit. Explaining the logic behind a specific trade decision to a portfolio manager is highly difficult. They also require large volumes of training data and significant computing power, usually demanding GPU hardware.

Equations:

A single neuron computes a weighted sum plus a bias: $z = \sum(x_i w_i) + b$. This value is passed through an activation function, such as the sigmoid

$f(z) = \left(\frac{1}{1 + e^{-z}} \right)$, to produce the node output.

Group Number: 15889

Features:

During training, input data moves forward through the layers (forward propagation) and a loss function measures the prediction error. The network then runs backpropagation, using the chain rule and gradient descent to adjust weights layer by layer and reduce the error on the next pass.

Guide:

- **Inputs:** scaled financial variables (e.g., historical price series, volatility measures, or economic indicators).
- **Outputs:** continuous values for regression, or class probabilities (via a softmax or sigmoid output layer) for classification.

Hyperparameters:

Parameters to tune include the network architecture (number of hidden layers and neurons per layer), `learning_rate`, `batch_size`, `epochs`, and the choice of optimiser (e.g., Adam or SGD).

Illustration:

A line graph of training and validation loss over epochs. The x-axis runs from 0 to 100 (epochs) and the y-axis shows mean squared error. Training loss falls steadily while validation loss levels off and eventually rises, marking the point where early stopping should intervene.

Journal:

Hutchinson, James M., Andrew W. Lo, and Tomaso Poggio. "A Nonparametric Approach to Pricing and Hedging Derivative Securities Via Learning Networks." *The Journal of Finance*, vol. 49, no. 3, 1994, pp. 851-889.

Keywords:

#Backpropagation, #ForwardPropagation, #GradientDescent, #ActivationFunction, #HiddenLayers.

Step 3: Technical Section

Hyperparameter Tuning Methodology:

Calibrating a model before live deployment requires tuning its hyperparameters. Unlike internal parameters (like network weights or decision tree split values) that the algorithm learns directly from training data, hyperparameters are set by the analyst beforehand. Examples include the maximum depth of a tree or the learning rate of an optimiser.

Because financial markets are highly noisy and prone to regime shifts, tuning functions as a primary defence against overfitting. Too much model flexibility leads to memorizing historical noise rather than finding tradeable patterns. To find the best configuration, we use Grid Search or Random Search alongside walk-forward cross-validation. Shuffling data for standard cross-validation is problematic in finance because future data leaks into the training sets, creating look-ahead bias. Walk-forward validation maintains chronological order, ensuring the selected parameters generalize to unseen periods.

- **Managing Overfitting:** Hyperparameters set limits on a model's complexity. For decision trees, restrict `max_depth` and `min_samples_split` to prevent the model from growing too complex and fitting noise.
- **Boundary Shapes:** Tuning an SVM involves a search over the kernel type (e.g., linear versus RBF) and `C`, the penalty parameter that controls the width of the soft margin and the number of allowed errors.
- **Training Diagnostics:** For neural networks, we configure structural parameters like hidden layers alongside optimizer settings such as learning rates. Checking validation error and stopping early allows us to stop training before the network begins overfitting.
- **Unsupervised Clustering:** K-Means has no labelled target variable, so traditional error metrics do not apply. Instead, `n_clusters` is selected using an elbow plot, and `n_init` is increased so that the algorithm runs from multiple starting points and is less likely to converge on a local minimum.
- **Covariance Estimation:** LDA assumes normal distributions and equal group covariance. When these assumptions fail, as is typical with financial features, we can apply shrinkage regularization to get more stable estimates of the covariance matrix.

Calibrating these parameters using validation folds ensures the backtest results translate into reliable trading strategies.

Step 4: Marketing Alpha

The Machine Learning Advantage in Quantitative Strategy:

Capturing alpha in modern markets is difficult using simple linear models that often fail when data is high-dimensional and non-stationary. The challenge is not gathering data, but separating signal from noise when dealing with many shifting indicators. These environments are suited for machine learning, which provides tools designed for high-dimensional, noisy environments, allowing quantitative desks to identify market states and model non-linear dynamics.

We choose algorithms according to the format of the data and the prediction goal, combining supervised models for directional forecasting with unsupervised clustering to identify regimes.

Uncovering Hidden Market Regimes:

Portfolio allocation benefits from grouping historical periods into distinct regimes. Because market regimes shift, models that assume static volatility often fail out of sample. We use K-Means clustering to group observations using historical volatility. Since K-Means is unsupervised, it clusters observations organically without needing supervisor labels. The model outputs categorical labels representing different market states, allowing us to adjust risk parameters dynamically. Being computationally cheap and scalable makes K-Means a solid foundation for identifying regimes.

Mastering Complexity and High-Dimensionality:

After identifying the market regime, we use SVM models to parse high-dimensional indicator spaces. The primary advantage of an SVM is its reliance on boundary-defining support vectors, which inherently resists market noise. Kernel functions enable the algorithm to find non-linear decision boundaries by projecting the features implicitly, bypassing the need to compute high-dimensional mappings directly.

For complex, time-varying interactions between macro indicators and asset prices, we use neural networks. The network's hidden layers automatically construct feature representations, minimizing manual feature creation. Weights are updated via backpropagation until training loss reaches a minimum. Though this setup models complex patterns, it demands large datasets and acts as a black box.

Preserving Interpretability and Risk Control:

However, institutional risk management requires transparency. We combine these black-box systems with transparent models to satisfy internal risk controls and regulatory standards.

We use decision trees for applications requiring auditability. The model splits data based on simple, sequential rules that can be mapped as a flowchart. Because trees are non-parametric and scale-invariant, they require no prior data transformation. This allows managers to trace any trading decision directly to specific rules.

Similarly, LDA offers a fast, probabilistic classifier. It projects high-dimensional inputs onto a lower-dimensional axis that maximizes class separation, outputting class probabilities via

Bayes' theorem. Beyond classification, the projected features can serve as a dimensionality reduction step. Given its long history in credit scoring and bankruptcy forecasting, we use its probability estimates to set rigid constraints for trade execution.

Step 5: Learn More

The following bibliography lists the academic research cited in this report alongside selected industry publications on machine learning in finance. The journal articles help readers explore the theory behind each model.

References:

Adcock, C. J., and N. Meade. "Using Parametric Classification Trees for Model Selection with Applications to Financial Risk Management." *European Journal of Operational Research*, vol. 259, no. 2, 2017, pp. 746-765. <https://www.sciencedirect.com/science/article/pii/S0377221716309018>.

Altman, Edward I. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *The Journal of Finance*, vol. 23, no. 4, 1968, pp. 589–609. <https://ideas.repec.org/a/bla/jfinan/v23y1968i4p589-609.html>.

Ammann, Manuel, and Michael Verhofen. "The Clustering of Hedge Funds." *Journal of Banking & Finance*, vol. 30, no. 11, 2006, pp. 3031–3059.

BlackRock Investment Institute. "Artificial Intelligence and Machine Learning in Asset Management." BlackRock, Oct. 2019.

"A Guide to Applying Machine Learning in Quantitative Finance | CQF." *Cqf.com*, Certificate in Quantitative Finance, 2026, pp. n. pag. <https://www.cqf.com/blog/guide-applying-machine-learning-quantitative-finance>.

Chui, Michael, et al. "An Executive's Guide to AI." McKinsey & Company, McKinsey Analytics, 2018. <https://www.mckinsey.com/capabilities/quantumblack/our-insights/an-executives-guide-to-ai>.

Hutchinson, James M., Andrew W. Lo, and Tomaso Poggio. "A Nonparametric Approach to Pricing and Hedging Derivative Securities Via Learning Networks." *The Journal of Finance*, vol. 49, no. 3, 1994, pp. 851-889. <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1994.tb00081.x>

"Non-Linear Stock Market Prediction with Support Vector Machines." *International Research Journal on Advanced Engineering Hub*, vol. 3, no. 3, 2025, pp. 615-621. <https://irjaeh.com/index.php/journal/article/view/576>.

Step 6: Comparing Models

Selecting a model requires balancing computational load, model transparency, and algorithmic assumptions against the properties of the data. The table below summarizes these characteristics for the five models covered in this report.

Model Feature	K-Means Clustering	Classification Trees	Linear Discriminant Analysis (LDA)	Support Vector Machine (SVM)	Neural Networks
Primary Learning Type	Unsupervised (Groups data without using pre-existing labels)	Supervised (Trained on past target outcomes)	Supervised (Trained on past target outcomes)	Supervised (Trained on past target outcomes)	Supervised (Trained on past target outcomes)
Interpretability	High: Centroids are easy to locate and describe mathematically.	Very High: Clear, logical rules that can be mapped as a simple flowchart.	High: Relies on clear linear formulas and outputs direct probabilities.	Moderate to Low: Clear in equation form, but non-linear boundaries are hard to visualize.	Very Low: An opaque model where hidden representation logic remains hidden.
Requires Feature Scaling?	Strictly Required: Distance-based calculation demands uniform scale.	Not Required: Split thresholds are invariant to feature scaling.	Recommended: Stabilises calculation when features have different ranges.	Strictly Required: Scaling is vital because margins are calculated geometrically.	Strictly Required: Equal feature scaling ensures gradient descent updates weights evenly.
Handles Non-Linear Data	Poorly: Struggles with non-spherical clusters or complex geometry.	Handles Well: Splits model non-linear curves without transformations.	Poorly: Limited to drawing linear decision boundaries.	Handles Well: Uses kernel functions to map data into higher dimensions.	Highly Effective: Non-linear activation functions naturally model complex functions.
Sensitivity to Outliers	High: Outliers pull centroid calculations away from main groups.	Low: Isolates outliers in small branches, protecting the rest of the model.	High: Extreme observations distort class means and covariance metrics.	Moderate to Low: Soft margins and slack terms ignore isolated anomalies.	Moderate: Prone to fitting noise unless controlled by early stopping.

Training Speed & Scalability	Fast: Scales well to large datasets and clusters quickly.	Fast: Rapid split computation, though deep trees consume memory.	Very Fast: Uses direct matrix algebra rather than iterative steps.	Very Slow: Training time scales poorly with the number of samples.	Very Slow: Demands GPU hardware and many training epochs.
Native Prediction Output	Discrete Label: Assigns each observation to a single cluster.	Probability and Class: Outputs both a label and its leaf-node probability.	Continuous Probability: Calculates class membership probability via Bayes' rule.	Rigid Class Only: Outputs a hard classification, requiring Platt scaling for probabilities.	Continuous Probability: Final layer outputs normalized probabilities.